

Q1. Provide an example to illustrate how this algorithm can be used to make decisions in a zero-sum game.

Ans. A zero-sum game is a situation where one player's gain is exactly balanced by the other player's loss. In simpler ~~ter~~ terms, the total 'payoff' is always zero - what one player wins, the other loses.

Decision-making in zero-sum games:-

To make optimal decisions in zero-sum games, algorithms often rely on strategies like the minimax algorithm, which is commonly used in turn-based games (like chess or tic-tack-toe) where two players compete.

The minimax algorithm involves:-

- i) Maximizing your gains while minimizing the opponent's gains.
- ii) Assuming that your opponent is also trying to minimize your gains.

• Examples: The Zero-Sum Game of "Odd or Even".

Consider a game called "Odd or Even," where:

- There are two players: Player A and Player B.
- Each player secretly chooses a number, either 1 or 2.
- Both players reveal their choices simultaneously.
 - ↳ if the sum of the two numbers is odd, Player A wins.
 - ↳ if the sum is even, Player B wins.
- The payoff is +1 for the winner and -1 for the loser (zero-sum).

• Decision Matrix:-

Player A \ Player B	1(B)	2(B)
1(A)	A: -1, B: +1	A: +1, B: -1
2(A)	A: +1, B: -1	A: -1, B: +1

- Using the Minimax Algorithm :-

Objectives: For Player A, the goal is to maximize the minimum gain (since Player B will play optimally to minimize Player A's gains). Similarly, Player B will try to minimize Player A's maximum possible gain.

Step 1: Analyze the Matrix for Player A

- if player A chooses 1:

- Against Player B's 1: Player A loses (-1)
- Against Player B's 2: Player A wins $(+1)$
- Minimum payoff for A if choosing 1: ~~to~~ -1

- if player A chooses 2:

- Against player B's 1: Player A wins $(+1)$
- Against player B's 2: Player A loses (-1)
- Minimum payoff for A if choosing 2: -1

In both cases, the worst-case scenario for Player A is a loss of -1 .

Step 2: Analyze the matrix for player B.

- if Player B choose 1:

- Against Player A's 1: Player B wins $(+1)$
- Against Player A's 2: Player B loses (-1)
- Maximum payoff for B if choosing 1: $+1$

- if Player B choose 2:

- Against Player A's 1: Player B loses (-1)
- Against Player A's 2: Player B wins $(+1)$
- Maximum payoff for B if choosing 2: $+1$

In both cases, the best-case scenario for Player B is gain of $+1$.

2. Does it optimize the minimax algorithm? Explain with an example how this technique reduces the number of nodes evaluated in the game tree.

Ans Yes, an optimization technique called Alpha-Beta Pruning is commonly used to improve the efficiency of the minimax algorithm. This technique significantly reduces the number of nodes evaluated in a game tree, allowing faster decision-making.

What is Alpha-Beta Pruning?

Alpha-beta Pruning is an optimization method that reduces the number of nodes the Minimax algorithm needs to evaluate. It works by "pruning" (like skipping) branches of the game tree that do not need to be explored because they cannot possibly affect the final decision.

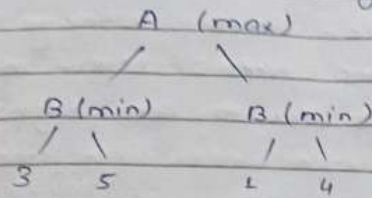
- Alpha represents the best maximum score that the maximizer (Player A) can guarantee so far.
- Beta represents the best minimum score that the minimizer (Player B) can guarantee so far.

How Does it Work :-

Alpha-Beta Pruning uses the following logic:

- As the Minimax algorithm explores the game tree, it keeps track of the best scores possible for both players using alpha and beta values.
- If at any point, the maximizer's best option (alpha) is greater than or equal to the minimizer's best option (beta), further exploration of that branch is stopped because it cannot influence the final decision.

Example: Consider a game tree:



- Without Pruning: The Minimax algorithm would evaluate all 4 leaf nodes (values 3, 5, 1, and 4) to find the best move.
- With alpha-beta Pruning:-
 - i) The algo. starts on the left side!
 - Evaluate 3 & 5 $\rightarrow$ max value = 5
 - Update Beta = 5
 - ii) Moves to the right side:-
 - Evaluate 1 $\rightarrow$ Updates Alpha = 1.
 - Since Alpha (1) < Beta (5), it continues to 4.
 - But since Alpha (5) is already better than 4, it prunes this branch.
- Result: Only 3 nodes (3, 5, 1) are evaluated instead of 4.

Thus, Alpha-beta pruning cuts off unnecessary branches, reducing the nodes evaluated and optimizing Minimax.

Q. How do algorithms differ in their approach to solving these types of games?

Ans Algorithms for solving zero-sum games, like minimax, and its optimizations (e.g. Alpha-Beta pruning), differ in how efficiently they explore the game tree and handle decision-making:

(i) Minimax Algorithm:-

- Aims to find the optimal move by exhaustively exploring all

possible game states.

- Assumes both players are playing optimally.
- Works well for small game trees but is inefficient for complex games like chess due to its high computational cost.

(ii) Alpha-Beta Pruning :-

- Optimizes Minimax by pruning branches that won't affect the final decision.
- Skips evaluating parts of the game tree if it's clear they won't influence the outcome, significantly reducing the number of nodes explored.
- Maintains the same results as Minimax but is much faster, making it suitable for deeper game trees.

(iii) Monte Carlo Tree Search (MCTS) :-

- Uses random simulations to explore the game tree, focusing on promising branches.
- Rather than exploring every possible move.
- Works well for games with large, complex trees where minimax is impractical.

4. Describe how you can represent the state space and we search strategies to find an optimal solution.

Ans To solve a game or decision problem, we can represent the state space and we search strategies to find an optimal solution

- State Space Representation :-
- The state space includes all possible configurations of the game.
- States :- snapshots of the game at a specific moment.
- Actions :- moves that change one state to another.

- Initial State: - The starting position.
- Goal State: - The winning or terminal condition.

Ex: In tic-tac-toe, states are board configurations, actions are placing 'x' or 'o', and the goal is to get three in a row.

Search Strategies:

(i) Minimax Algo: -

- Used in two-player games to maximize a player's score while minimizing the opponent's.
- Explores all possible moves to find the optimal strategy.

(ii) Alpha-Beta Pruning: -

- ↳ An optimization of Minimax that prunes irrelevant branches, reducing the number of states to explore.
- ↳ Improve efficiency without changing the outcome.

(iii) Breadth-First Search (BFS): -

- Explores all moves level by level.
- Guarantees finding the optimal solution but can be slow for large state spaces.

(iv) Depth-First Search (DFS): -

- Explores moves deeply along a branch backtracking.
- Uses less memory but may not always find the optimal solution first.

Q. What are the key differences in the approach and the computational complexities involved?

Ans. Key differences and computational complexities:-

(i) Minimax Algorithm: -

- Approach - Exhaustively explores all possible game states to find the optimal move for both players.
- Complexity: $O(b^d)$, where b = branch factor and d = depth.
- Limitation: - Computationally expensive for large game (e.g. chess).

(ii) Alpha-Beta Pruning -

- Approach: - Optimize Minimax by skipping branches that can't affect the final outcome.
- Complexity: - Base case $O(b^{d/2})$
- Advantage: - More efficient.

(iii) Breadth-First Search (BFS): -

- Approach - Explores all nodes level by level until it finds a solution.
- Complexity - $O(b^d)$ in time and space.
- Limitation - more memory required.

(iv) Depth-First Search (DFS): -

- Approach - Explores nodes deeply along one branch before backtracking.
- Complexity: - $O(b^d)$ in time but uses only $O(d)$ space.
- Limitation: - May not find the optimal solution first in non-uniform trees.

x ————— x ————— x ————— x ————— x